

day 15

day 1 node

📌 01. Obsidian Notes: Introduction to Backend & Node.js

المفهوم الأساسي 💡 📌

الـ Backend هو المحرك الخلفي المنظم للشبكة، الذي يعالج البيانات، ويتصل بقواعد البيانات، ويتحكم في عمليات الصلاحيات والأمان (Authentication & Authorization).

🌐 1. Client-Server Architecture & HTTP Basics

- **Client (Front-end):** HTML, CSS, JavaScript التي تحول طلبات المستخدم إلى Requests.
- **Server (Back-end):** ويعالجها من خلال قواعد البيانات برمجياً، ثم **IP Address** جهاز/نظام يستقبل الطلبات عبر الـ **HTTP Response** يعيد.
- **IP Address (IPv4 / IPv6):** العنوان الفريد لكل جهاز على شبكة الإنترنت.
- **DNS (Domain Name System):** الخاص IP إلى رقم (مثل google.com) النظام المسئول عن تحويل النطاق بالسيرفر.

🔄 2. RESTful APIs & HTTP Verbs

يعتمد على المبادئ التالية APIs هو أسلوب معماري لبناء الـ REST (Representational State Transfer):

1. الفصل التام بين الـ Client والـ Server.
2. **Stateless:** يجب أن يحتوي على جميع البيانات request لا يحفظ السيرفر أي حالة سابقة للمستخدم بين الطلبات؛ كل المطلوبة للطلب.
3. التعبير عن الموارد بأرقام وأسماء نصوص (Resources) وليس أفعال (Verbs).

🔧 أشهر الـ HTTP Methods والـ Status Codes:

Method	Endpoint Contoh	الوظيفة
GET	/users	جلب وقراءة البيانات من السيرفر.
POST	/users	إنشاء وتخزين عنصر جديد في السيرفر (بيانات في Body).
PUT	/users/:id	تعديل وتبديل العنصر كاملاً في السيرفر.
PATCH	/users/:id	تعديل جزء محدد فقط من بيانات العنصر.

Method	Endpoint Contoh	الوظيفة
DELETE	/users/:id	حذف العنصر من السيرفر.

أهم الـ HTTP Status Codes: 🚩

- **200 OK:** تم تنفيذ الطلب بنجاح.
- **201 Created:** (مع POST) تم إنشاء عنصر جديد بنجاح.
- **400 Bad Request:** خطأ في البيانات المبعوثة من العميل.
- **401 Unauthorized / 403 Forbidden:** خطأ في تسجيل الدخول أو عدم وجود صلاحية.
- **404 Not Found:** المورد المطلوبة أو المسار غير موجود.
- **500 Internal Server Error:** حدث خطأ داخل كود السيرفر نفسه.

3. Node.js & Express.js Core Framework

- **Node.js:** يتيح تشغيل كود جافاسكريبت خارج V8 مبني على (JavaScript Runtime Environment) محرك تشغيل Backend المتصفح كـ.
- **Express.js:** Middleware والـ Routing يسهل التعامل مع الـ (Minimal Framework) إطار عمل خفيف وسريع.
- **Middlewares:** أو Request وتستطيع تعديل الـ Response وإرسال الـ Request دوال وسيطة تنفذ بين استقبال الـ فحص التوكين أو معالجة الأخطاء.

4. MongoDB & Mongoose (BSON & Schemas)

- **MongoDB:** الداعمة لـ **BSON** (Binary JSON) تخزين البيانات بصيغة (NoSQL) قاعدة بيانات غير علائقية ObjectId و Data types المعالجة والسرعة البحث والمعالجة.
- **Mongoose:** وقواعد على البيانات قبل حفظها Validation وإضافة Schema لتحديد الـ (ODM) مكتبة.

5. Security & Authentication Fundamentals

1. **Hashing (Bcrypt):** لمنع هجمات Salt لتحويل الباسورد إلى سلسلة مشفرة باستخدام (Irreversible) عملية غير عكسية: التخمين.
2. **JWT (JSON Web Token):** تتكون من 3 أجزاء Stateless طريقة توثيق: Header.Payload.Signature .
3. **Security Middlewares:**
 - **Helmet:** XSS و Clickjacking ضد هجمات HTTP Headers حماية.
 - **Express-Rate-Limit:** DoS / Brute force منعاً لهجمات IP تحديد عدد الطلبات لكل.
 - **Mongo-Sanitize:** NoSQL Injection تنظيف المدخلات لمنع.

أسئلة إنترفيو متوقعة (Interview Q&A) ?

❓ **Q1: Why does `setImmediate` execute before `setTimeout(fn, 0)` inside an I/O cycle (like `fs.readFile`)?**

الإجابة: لأن الدورة الروحية لـ Event Loop بعد انتهاء مرحلة الـ I/O Polling تذهب مباشرة لمرحلة الـ Check Phase (المسئولة عن `setImmediate`) قبل العودة لدورة جديدة لبدء مرحلة Timers Phase (المسئولة عن `setTimeout`).

❓ Q2: What is the primary difference between `process.nextTick()` and `Promise.then()` ?

الإجابة: كلاهما Microtasks، ولكن `process.nextTick()` لها أولوية تنفيذ أعلى مطلقاً على الـ Microtask Queue؛ حيث يتم تفرغ طابور `nextTick` كاملاً قبل الانتقال لتنفيذ أي `Promise`.